

TYPESCRIPT - Aula 15/08/2024

Introdução ao TypeScript

Objetivos da Aula

1. **Compreender o que é TypeScript e suas vantagens.**
2. **Aprender os fundamentos da linguagem TypeScript.**
3. **Aplicar TypeScript em um projeto básico.**
4. **Preparar a base para integração com ReactJS e Vite nas próximas aulas.**

1. O Que é TypeScript?

TypeScript é um superconjunto tipado de JavaScript que compila para JavaScript puro. Ele adiciona tipagem estática à linguagem, o que ajuda a identificar erros mais cedo e melhora a manutenção do código.

Vantagens do TypeScript:

- **Tipagem Estática:** Ajuda a encontrar erros durante o desenvolvimento.
- **IntelliSense:** Melhora a autocompletar e a navegação no código.
- **Documentação:** Tipos funcionam como documentação para o código.
- **Orientação a Objetos:** Suporte a classes e interfaces.

2. Configuração Inicial

Passos para Configuração:

1. **Instalação do TypeScript:**

   ```
   npm install -g typescript
   ```

2. **Criar um Projeto TypeScript:**

   - Crie uma pasta para o projeto e inicialize com `npm`:

     ```
     mkdir meu-projeto-typescript
     cd meu-projeto-typescript
     npm init -y
     ```

- Instale o TypeScript como dependência de desenvolvimento:

```
npm install typescript --save-dev
```

- Crie um arquivo `tsconfig.json` com o comando:

```
npx tsc --init
```

- Estrutura básica do `tsconfig.json`:

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "commonjs",
    "strict": true,
    "esModuleInterop": true
  },
  "include": ["src/**/*"]
}
```

- Compilar o arquivo TypeScript

Para compilar o arquivo `index.ts`, use o seguinte comando no terminal:

```
tsc index.ts
```

Isso gerará um arquivo JavaScript chamado `index.js` na mesma pasta.

- Executar o arquivo compilado

Depois de compilar, você pode rodar o arquivo JavaScript gerado com Node.js:

```
node index.js
```

O terminal deverá exibir:

```
Olá, TypeScript!
```

3. Fundamentos do TypeScript

Tipos Básicos:

- **String, Number, Boolean, Array, Tuple, Enum, Any, Void, Null e Undefined**

  Exemplos:

```
let nome: string = "João";
let idade: number = 30;
let ativo: boolean = true;
```

```
let numeros: number[] = [1, 2, 3];
let tupla: [string, number] = ["Alice", 25];

enum Cor {
  Vermelho,
  Verde,
  Azul
}

let cor: Cor = Cor.Verde;
```

Funções:

- **Definição de Tipos de Parâmetros e Retornos**

  Exemplos:

```
function cumprimentar(nome: string): string {
  return `Olá, ${nome}`;
}

function somar(a: number, b: number): number {
  return a + b;
}
```

Interfaces e Tipos:

- **Definindo Estruturas e Tipos Personalizados**

  Exemplos:

```
interface Pessoa {
  nome: string;
  idade: number;
}

let pessoa: Pessoa = {
  nome: "Ana",
  idade: 28
};

type ID = string | number;

let idUsuario: ID = "123";
```

Classes:

- **Definição de Classes e Herança**

Exemplos:

```
class Animal {
  nome: string;

  constructor(nome: string) {
    this.nome = nome;
  }

  falar(): void {
    console.log(`${this.nome} faz um barulho.`);
  }
}

class Cachorro extends Animal {
  latir(): void {
    console.log(`${this.nome} late.`);
  }
}

let cachorro = new Cachorro("Buddy");
cachorro.falar();
cachorro.latir();
```

Null e Undefined

Em TypeScript, `null` e `undefined` são dois valores especiais que representam a ausência de um valor, mas têm usos e comportamentos distintos. Vamos explorar o conceito de cada um, como são utilizados, e como o TypeScript lida com eles.

`undefined`

Conceito:

- `undefined` significa que uma variável foi declarada, mas ainda não recebeu um valor.
- Também é o valor retornado quando se tenta acessar uma propriedade de um objeto que não existe.

Exemplos:

```
let valorIndefinido: string | undefined;
console.log(valorIndefinido); // undefined

function buscarPropriedade(obj: { [key: string]: any }, chav
e: string) {
  return obj[chave];
}
```

```
const obj = { nome: "João" };
console.log(buscarPropriedade(obj, "idade")); // undefined
```

`null`

Conceito:

- `null` é um valor explícito que indica a ausência de qualquer valor ou objeto.
- É comumente utilizado quando se deseja intencionalmente limpar ou inicializar uma variável ou propriedade sem valor.

Exemplos:

```
let valorNulo: string | null = null;
console.log(valorNulo); // null

let objeto: { nome: string | null } = { nome: null };
console.log(objeto.nome); // null
```

Diferenças entre `null` e `undefined`

- `undefined`: Normalmente, é usado pelo JavaScript quando uma variável é declarada mas não inicializada, ou quando uma função não retorna explicitamente um valor.
- `null`: Usado explicitamente pelos desenvolvedores para indicar a ausência de um valor.

Exemplo de Comparação:

```
let x: undefined = undefined;
let y: null = null;

console.log(x == y);  // true (comparação de igualdade froux
a)
console.log(x === y); // false (comparação de igualdade estri
ta)
```

No exemplo acima, `x == y` retorna `true` porque JavaScript considera `null` e `undefined` como iguais em valor (embora de tipos diferentes). Porém, `x === y` retorna `false` porque a comparação estrita verifica o tipo e o valor.

Como TypeScript Lida com `null` e `undefined`

Em TypeScript, os valores `null` e `undefined` são tratados como tipos distintos. Por padrão, uma variável que pode ser `null` ou `undefined` precisa ser explicitamente tipada para aceitar esses valores, a menos que o modo `strictNullChecks` esteja desativado.

Exemplo com `strictNullChecks`:

```
let valor: string;
valor = null; // Erro em strict mode: Type 'null' is not assi
gnable to type 'string'.

let valorOpcional: string | null = null; // OK
valorOpcional = "texto";
```

Exemplo de Função que Pode Retornar `null` ou `undefined`:

```
function buscarValor(mapa: Map<string, string>, chave: strin
g): string | undefined {
  return mapa.get(chave);
}

const mapa = new Map([["chave1", "valor1"]]);

let resultado = buscarValor(mapa, "chave1"); // "valor1"
resultado = buscarValor(mapa, "chave2"); // undefined
```

Conclusão

- `undefined` é utilizado quando uma variável é declarada mas não inicializada.
- `null` é utilizado explicitamente para indicar a ausência de valor.
- O TypeScript força o desenvolvedor a lidar com esses casos explicitamente, ajudando a evitar erros comuns em JavaScript.

Com esses conceitos e exemplos, podemos entender como e quando usar `null` e `undefined` em TypeScript, além de como esses valores interagem com o sistema de tipos da linguagem.

---

4. Exercícios Práticos

Exercício 1: Tipagem Básica

Exemplo:
Crie um arquivo
`index.ts` e defina variáveis para diferentes tipos de dados, como string, número, e booleano. Exiba esses valores no console.

```
// index.ts
```

```
let nome: string = "João";
let idade: number = 30;
let ativo: boolean = true;

console.log(`Nome: ${nome}`);
console.log(`Idade: ${idade}`);
console.log(`Ativo: ${ativo}`);
```

Seu Exercício:

1. Crie um arquivo `dados.ts`.
2. Defina variáveis para pelo menos três tipos diferentes de dados, como `string`, `number`, e `boolean`.
3. Exiba essas variáveis no console.

Dicas:

- Utilize diferentes tipos de dados como `string`, `number`, `boolean`, e `array`.
- Experimente também o tipo `any` para variáveis que podem ter valores variados.

Exemplo:

```
// dados.ts

let cidade: string = "São Paulo";
let temperatura: number = 25.5;
let chovendo: boolean = false;

console.log(`Cidade: ${cidade}`);
console.log(`Temperatura: ${temperatura}`);
console.log(`Chovendo: ${chovendo}`);
```

Exercício 2: Funções e Interfaces

Exemplo:
Crie uma interface para representar um produto com as propriedades `nome`, `preço`, e `emEstoque`. Escreva uma função que recebe um produto e retorna uma mensagem sobre a disponibilidade.

```
// interface.ts

interface Produto {
  nome: string;
  preco: number;
  emEstoque: boolean;
}

function verificarDisponibilidade(produto: Produto): string {
```

```
  if (produto.emEstoque) {
    return `O produto ${produto.nome} está disponível por R
$${produto.preco}.`;
  } else {
    return `O produto ${produto.nome} não está disponível no
momento.`;
  }
}

const produto1: Produto = {
  nome: "Cadeira",
  preco: 150.00,
  emEstoque: true
};

console.log(verificarDisponibilidade(produto1));
```

Seu Exercício:

1. Crie um arquivo `produto.ts`.
2. Defina uma interface para representar um livro com as propriedades `titulo`, `autor`, e `anoPublicacao`.
3. Escreva uma função que recebe um livro e retorna uma mensagem informando o título e o ano de publicação.

Dicas:

- Utilize propriedades adequadas para a interface.
- A função deve construir uma mensagem informativa baseada nas propriedades do livro.

Exemplo:

```
// produto.ts

interface Livro {
  titulo: string;
  autor: string;
  anoPublicacao: number;
}

function exibirInformacoes(livro: Livro): string {
  return `O livro "${livro.titulo}" foi escrito por ${livro.a
utor} e publicado em ${livro.anoPublicacao}.`;
}

const livro1: Livro = {
  titulo: "1984",
  autor: "George Orwell",
```

```
  anoPublicacao: 1949
};

console.log(exibirInformacoes(livro1));
```

Exercício 3: Classes e Herança

Exemplo:
Crie uma classe
`Veiculo` com as propriedades `marca` e `modelo`. Crie uma classe `Carro` que herda de `Veiculo` e adiciona uma propriedade `tipoCombustivel`. Instancie um objeto `Carro` e exiba suas propriedades no console.

```
// index.ts

class Veiculo {
  marca: string;
  modelo: string;

  constructor(marca: string, modelo: string) {
    this.marca = marca;
    this.modelo = modelo;
  }

  exibirInfo(): void {
    console.log(`Marca: ${this.marca}, Modelo: ${this.modelo}
`);
  }
}

class Carro extends Veiculo {
  tipoCombustivel: string;

  constructor(marca: string, modelo: string, tipoCombustivel:
string) {
    super(marca, modelo);
    this.tipoCombustivel = tipoCombustivel;
  }

  exibirInfoCompleta(): void {
    console.log(`Marca: ${this.marca}, Modelo: ${this.model
o}, Tipo de Combustível: ${this.tipoCombustivel}`);
  }
}

const meuCarro = new Carro("Toyota", "Corolla", "Flex");
meuCarro.exibirInfoCompleta();
```

Seu Exercício:

1. Crie um arquivo `veiculo.ts`.
2. Defina uma classe `Pessoa` com propriedades `nome` e `idade`.
3. Crie uma classe `Aluno` que herda de `Pessoa` e adiciona uma propriedade `curso`.
4. Instancie um objeto `Aluno` e exiba suas propriedades no console.

Dicas:

- Utilize o método `super` para herdar propriedades da classe pai.
- Adicione um método na classe filha para exibir todas as propriedades.

Exemplo:

```
// veiculo.ts

class Pessoa {
  nome: string;
  idade: number;

  constructor(nome: string, idade: number) {
    this.nome = nome;
    this.idade = idade;
  }

  exibirInformacoes(): void {
    console.log(`Nome: ${this.nome}, Idade: ${this.idade}`);
  }
}

class Aluno extends Pessoa {
  curso: string;

  constructor(nome: string, idade: number, curso: string) {
    super(nome, idade);
    this.curso = curso;
  }

  exibirInformacoesCompleta(): void {
    console.log(`Nome: ${this.nome}, Idade: ${this.idade}, Cu
rso: ${this.curso}`);
  }
}

const aluno1 = new Aluno("Mariana", 21, "Engenharia de Comput
ação");
aluno1.exibirInformacoesCompleta();
```

Esses exercícios ajudará a praticar a sintaxe básica do TypeScript e a aplicar conceitos fundamentais, preparando-os para utilizar TypeScript com ReactJS e Expo nas próximas aulas. Se precisar de mais alguma coisa, é só avisar!

> Professor Hete Caetano